ECE 220 Computer Systems & Programming

Lecture 12 — Recursion

October 6, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Self-Reference Problem

Quicksort's last step was "sort each half the same way."
A function that calls itself — why does that even work?

You already know the machinery:
every call gets a **fresh activation record**
(Lecture 8) — even a call to yourself.

Today's Two Questions

A recursive function is a recurrence your computer can run.

1. What makes a self-calling function *terminate* — and compute the right answer?
2. What actually happens on the run-time stack while it runs?

Welcome back from Midterm 1. First a sorting warm-up — then functions meet mirrors.

Warm-up: Implement Insertion Sort

Last lecture's card-sorting idea, in code:

- Take `array[i]`; scan the sorted prefix right-to-left
- Shift bigger elements right; track the _____
- Drop the taken item into the gap

```
void insertion_sort(int array[]) {
    int i, j, temp, empty_idx;
    for (i = 1; i < SIZE; i++) {
        /* take array[i]; remember the gap index  */

        /* scan the sorted part right-to-left:    */
        /* shift larger elements right,           */
        /* moving the gap left as you go          */

        /* insert the taken item into the gap     */

    }
}
```

Recursion = Self-Call + Base Case

A **recursive function** solves its task by calling itself on *smaller pieces* of the problem.

The math:

```
RunningSum(1) = 1
RunningSum(n) =
  n + RunningSum(n-1)
```

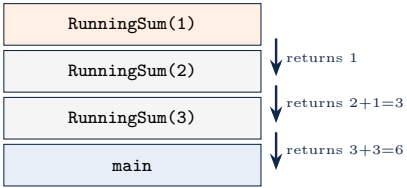
- At least one **base case** that ends the process: here _____
- The recursive call must move *toward* it

The code:

```
int RunningSum(int n) {
    if (n == 1)
        return 1;                    /* base case
      */
    else
        return n + RunningSum(n - 1);
}
```

What Happens on the Stack

`RunningSum(3)`:



- Each call pushes a **fresh frame** — its own `n`, its own return address
- Deepest moment:

  _____
- The pending `n + ...` of every caller waits in its frame until the callee returns

No base case ⇒ frames pile up until the stack is gone — "stack overflow" is a literal description.

In-Class Problem: Fibonacci

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

$F_n = F_{n-1} + F_{n-2}, \quad F_0 = 0, F_1 = 1$

- How many base cases?

  _____

- Total calls to compute `Fib(5)`?

  _____

-

```
int Fibonacci(int n) {   /* n >= 0 */
    /* base cases: which n need no recursion? */

    /* recursive case: the recurrence itself  */

}
```

Fibonacci with a Lookup Table

```
int table[100];           /* main sets all to -1 */

int fibonacci(int n) {    /* 0 <= n < 100 */
    if (table[n] != -1)
        return table[n];          /* already known */
    if (n == 0)
        table[n] = 0;
    else if (n == 1)
        table[n] = 1;
    else
        table[n] = fibonacci(n - 1) + fibonacci(n - 2);
    return table[n];
}
```

- Remember every answer the first time you compute it (**memoization**)
- Calls for `fib(5)` now?

- Same recurrence, exponential $\rightarrow$ linear — bought with one global array

In-Class Problem: Recursive Binary Search

Last lecture's loop, reborn as a recurrence:

- Base cases?

  _____

- The recursive call searches

  _____

- Why must `start` and `end` be parameters now?

```
int binary(int array[], int item, int start, int
     end) {
    /* base case: the region is empty */

    /* probe the middle; found? done  */

    /* recurse into the correct half  */

}
```

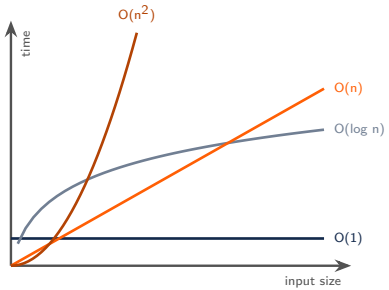
Quicksort, Completed

```
    int p;
    if (start >= end)
        return;              /* base: 0 or 1
     element */
    p = partition(array, start, end);
    quicksort(array, start, p - 1);
    quicksort(array, p + 1, end);
}
```

(`partition` picks a pivot, moves smaller elements left of it, returns the pivot's final index — full code in `quicksort.c`.)

- Base case: _____
- Two recursive calls — the recursion *tree* from last lecture's picture is exactly the call tree
- Last lecture's promise, kept: "sort each side by the same method" is one line, twice

A Sneak Peek at Big-O



linear search	_____
binary search	_____
bubble / insertion sort	_____
quicksort (typical)	O(n log n)

A vocabulary preview — ECE 374 makes it precise. For now: the *shape* of the curve is what survives when n gets big.

Summary & What's Next

Today

- **Recursion**: a base case, plus a self-call on a smaller problem — the code is the recurrence
- **On the stack**: every call gets a fresh frame; no base case means literal stack overflow
- **At work**: Fibonacci (and its exponential tree, tamed by a lookup table), recursive binary search, quicksort completed
- **Big-O**: the growth curve is the algorithm's signature

Next lecture

- Recursion at full power: systematic trial and error — backtracking

Code from today: `insertion.c`, `runsum.c`, `fib.c`, `fibtable.c`, `rbsearch.c`, `quicksort.c`.